

United International University

Department of CSE

Course Code: CSE 4326

Course Name: Microprocessor and Microcontroller Laboratory

Experiment no. 4

Experiment Name: Introduction to Raspberry Pi

Submitted by (Group no. 7):

Name: Nafis Iqbal Neloy

Student ID: 0112330617

Section: M

Date of Performance: 7th September, 2026

Date of Submission: 14th September, 2026

Objective:

The objective of this experiment is to set up a Raspberry Pi to build sensor-based systems. First, we will interface a gas sensor with the Raspberry Pi to monitor gas-level data in ppm and store the collected data for further analysis. Next, we will interface an ultrasonic sensor to measure the distance of an object and display the distance in the terminal. Finally, we will connect an LED with the Raspberry Pi to provide a visual indication when an object comes very close to the ultrasonic sensor. Through these tasks, we learn how to interface sensors with the Raspberry Pi, process sensor data using Python, store sensor readings, and control output devices based on real-time sensor measurements.

Apparatus/Instruments used in the Lab:

- Raspberry Pi 5
- Gas Sensor
- Ultrasonic Sensor
- LED
- Resistors
- Breadboard
- Jumper Wires
- Raspberry Pi Power Supply
- Computer/Monitor for programming and terminal output

Theory:

The Raspberry Pi is a single-board computer that can interface with various electronic components through its GPIO pins. In this experiment, the Raspberry Pi is used to collect and process data using Python and to control an output device based on real-time measurements.

A gas sensor detects the presence and level of gases by producing an electrical signal that changes according to the surrounding gas concentration. The sensor readings can be processed by the Raspberry Pi and represented as gas concentration values in parts per million (ppm). The collected readings can also be stored in a file for future analysis and record keeping.

An ultrasonic sensor measures the distance of an object by transmitting a high-frequency sound wave and measuring the time taken for the reflected wave to return. The Raspberry Pi uses this time measurement to determine the distance of the object and displays the result in the terminal.

Solution of the mentioned problems:

Problem 1: Gas Sensor Interfacing and Data Logging

The gas sensor was connected to the Raspberry Pi to monitor gas-level data. The sensor output was read by the Raspberry Pi and processed using Python. The gas concentration data was represented in parts per million (PPM). The measured data was also stored in an Excel file along with the corresponding time for future analysis.

Procedure:

1. Connect the gas sensor to the Raspberry Pi.
2. Configure the required GPIO/ADC connection for reading the sensor output.
3. Use Python to continuously read the gas sensor value.
4. Convert the sensor reading into gas concentration in PPM.
5. Display the PPM value in the terminal.
6. Record the PPM value along with the corresponding time.
7. Store the collected data in an Excel file.

1(a) Python Code to Determine Gas Volume in PPM

The following Python code reads the gas sensor value and determines the gas concentration in PPM

```
from gpiozero import DigitalInputDevice
from time import sleep

# Gas sensor digital output connected to GPIO 17
gas = DigitalInputDevice(17)

print("Gas Sensor Monitoring Started...")
print("Press Ctrl+C to stop.")

try:
    while True:
        # Many gas sensor modules output LOW when gas is detected
        if gas.value == 0:
            print("Gas Detected")
        else:
            print("Normal")

        sleep(2)

except KeyboardInterrupt:
    print("\nGas monitoring stopped.")
```

1(b) Python Code to Store Gas Data in an Excel File

```
from gpiozero import DigitalInputDevice
from openpyxl import Workbook, load_workbook
from datetime import datetime
from time import sleep
import os

# Gas sensor digital output connected to GPIO 17
gas = DigitalInputDevice(17)

# Excel file name
filename = "Gas_Data.xlsx"

# Create Excel file if it does not already exist
if not os.path.exists(filename):
    workbook = Workbook()
    sheet = workbook.active
    sheet.title = "Gas Data"

    # Add column headings
    sheet.append(["Time", "Status"])

    workbook.save(filename)

print("Gas Data Logging Started...")
print("Press Ctrl+C to stop.")

try:
    while True:
        # Read gas sensor status
        if gas.value == 0:
            status = "Gas Detected"
        else:
            status = "Normal"

        # Get current time
        current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # Display result in terminal
        print(current_time, "-", status)

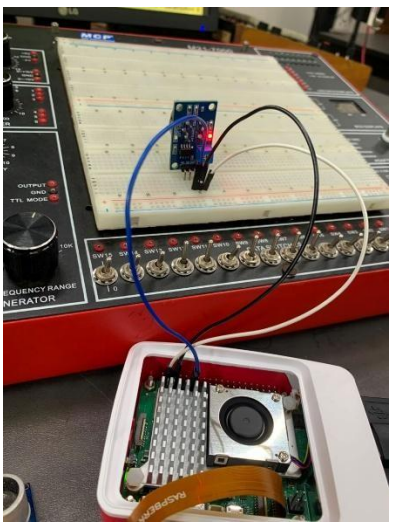
        # Open Excel file
        workbook = load_workbook(filename)
        sheet = workbook["Gas Data"]

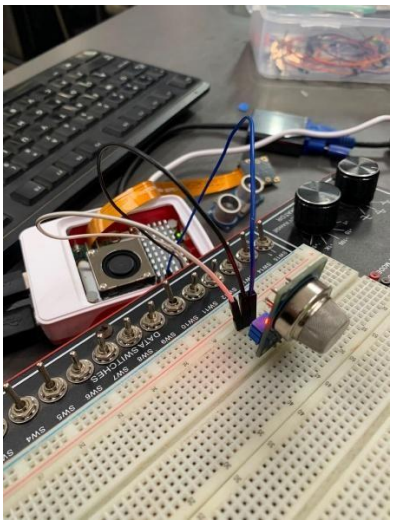
        # Store time and sensor status
        sheet.append([current_time, status])

        # Save Excel file
        workbook.save(filename)

        sleep(2)

except KeyboardInterrupt:
    print("\nData logging stopped.")
    print("Gas data saved to:", os.path.abspath(filename))
```





	A	B	C	D
1	Time	Status		
2	2026-08-03 11:09:53	Normal		
3	2026-08-03 11:09:55	Normal		
4	2026-08-03 11:09:57	Normal		
5	2026-08-03 11:09:59	Normal		
6	2026-08-03 11:10:01	Normal		
7	2026-08-03 11:10:03	Normal		
8	2026-08-03 11:10:06	Normal		
9	2026-08-03 11:10:08	Normal		
10	2026-08-03 11:10:10	Normal		
11	2026-08-03 11:10:12	Normal		
12	2026-08-03 11:10:14	Normal		
13	2026-08-03 11:10:16	Normal		
14	2026-08-03 11:10:18	Normal		
15	2026-08-03 11:10:20	Normal		
16	2026-08-03 11:10:22	Normal		
17	2026-08-03 11:10:24	Normal		
18	2026-08-03 11:10:26	Normal		
19	2026-08-03 11:10:28	Normal		
20	2026-08-03 11:10:30	Normal		
21				

Figure 1: Circuit Connection of Gas Sensor and its output in Excel with Raspberry Pi

Problem 2: Ultrasonic Sensor and LED

In this problem, an ultrasonic sensor was connected to the Raspberry Pi to measure the distance of an object. The Raspberry Pi calculates the distance based on the time taken for the ultrasonic signal to travel to the object and return to the sensor. The measured distance is displayed in the terminal in centimeters. An LED is also connected to the Raspberry Pi as a visual indicator. When an object comes within a predefined distance of the sensor, the LED turns on, and when the object moves beyond the threshold distance, the LED turns off.

Procedure:

1. Connect the ultrasonic sensor to the Raspberry Pi.
2. Connect the LED to a GPIO pin through a suitable resistor.
3. Configure the trigger and echo pins of the ultrasonic sensor.
4. Measure the distance of an object continuously.
5. Display the measured distance in the terminal.
6. Set a threshold distance for detecting a nearby object.
7. Turn the LED on when the object is within the threshold and turn it off otherwise.

Related Code:

```
from gpiozero import DistanceSensor, LED
from time import sleep
import warnings

# Hide gpiozero warnings
warnings.filterwarnings("ignore")

# HC-SR04 connections
TRIG_PIN = 23
ECHO_PIN = 24

# LED connection
LED_PIN = 17

# Initialize sensor
sensor = DistanceSensor(
    echo=ECHO_PIN,
    trigger=TRIG_PIN,
    max_distance=2
)

# Initialize LED as active-low
led = LED(LED_PIN, active_high=False)

# Make sure LED starts OFF
led.off()

print("Ultrasonic Sensor Started...")
print("Press Ctrl+C to stop.\n")

try:
    while True:
        # Distance in centimeters
        distance = sensor.distance * 100

        # Ignore invalid/no-echo readings
        if distance >= 199:
            print("No object detected")
            led.off()

        else:
            print(f"Distance: {distance:.2f} cm")

            # LED ON only if object is within 10 cm
            if distance <= 10:
                led.on()
                print(">>> Object is VERY CLOSE! LED ON")
            else:
                led.off()
                print(">>> Safe Distance. LED OFF")

        sleep(0.5)

except KeyboardInterrupt:
    led.off()
    print("\nProgram Stopped")
```

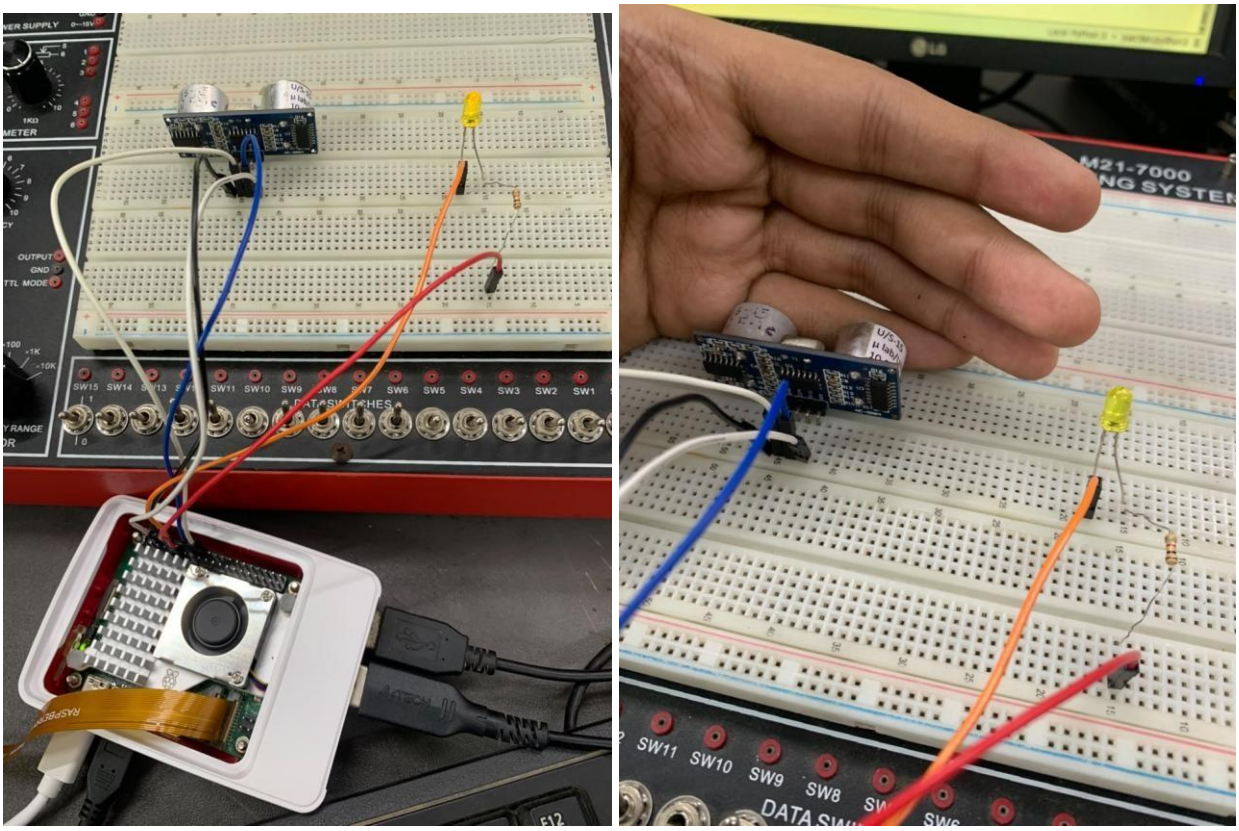


Figure 2.1: Circuit Connection of Ultrasonic Sensor and LED with Raspberry Pi

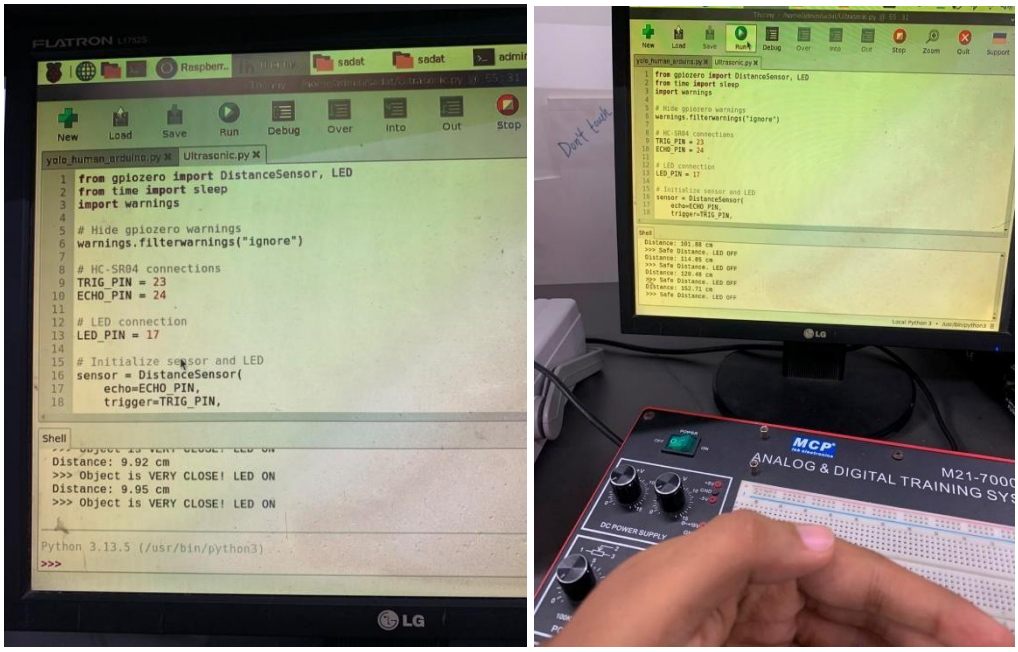


Figure 2.2: Output of Ultrasonic Sensor Distance Measurement and LED Control

Discussion:

The experiment successfully demonstrated how a Raspberry Pi can be used to interface with sensors, process real-time data, and control output devices. In the first problem, the gas sensor was used to monitor gas concentration, and the sensor readings were processed to determine the gas volume in PPM. The PPM data was also recorded along with time information and stored in an Excel file for future analysis.

In the second problem, the ultrasonic sensor successfully measured the distance of objects and displayed the readings in the terminal. The LED was controlled according to a predefined distance threshold, turning on when an object came very close to the sensor and remaining off when the object was farther away. Some variations in ultrasonic readings were observed when the sensor did not receive an echo, which can occur due to the position or distance of the object.

Overall, the experiment provided practical experience in GPIO interfacing, gas concentration measurement, PPM data processing, Excel data logging, distance measurement, and controlling an output device using Python on the Raspberry Pi.